Solution: Maximum Ribbon Cut

Introduction

We are given a ribbon of length 'n' and a set of possible ribbon lengths. We need to cut the ribbon into the maximum number of pieces that comply with the above-mentioned possible lengths. Write a method that will return the count of pieces.

Example 1:

```
n: 5
Ribbon Lengths: {2,3,5}
Output: 2
Explanation: Ribbon pieces will be {2,3}.
```

Example 2:

```
n: 7
Ribbon Lengths: {2,3}
Output: 3
Explanation: Ribbon pieces will be {2,2,3}.
```

Example 3:

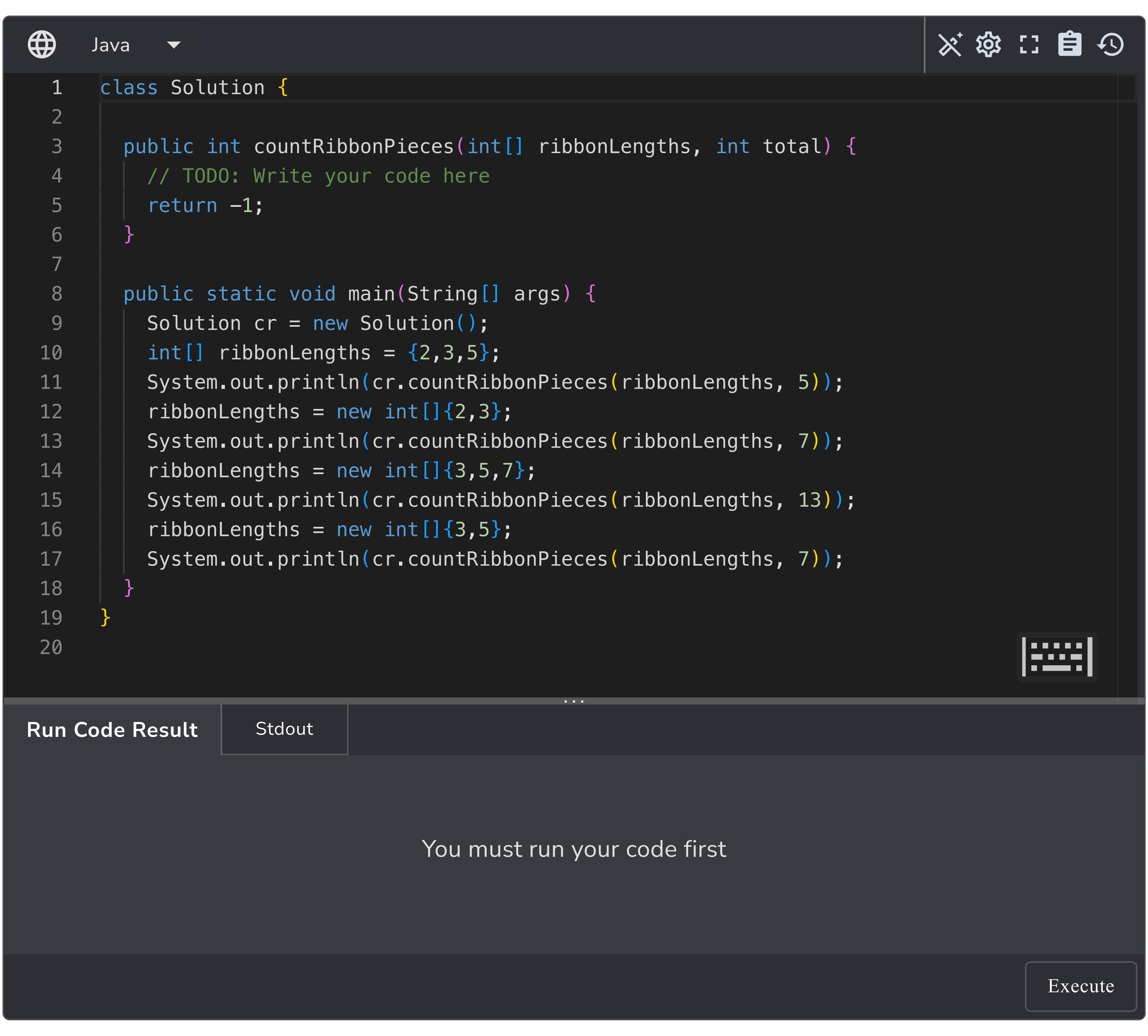
```
n: 13
Ribbon Lengths: {3,5,7}
Output: 3
Explanation: Ribbon pieces will be {3,3,7}.
```

Problem Statement

Given a number array to represent possible ribbon lengths and a total ribbon length 'n,' we need to find the maximum number of pieces that the ribbon can be cut into.

Try it yourself

Try solving this question here:



```java
class Solution {

  public int countRibbonPieces(int[] ribbonLengths, int total) {
    // TODO: Write your code here
    return -1;
  }

  public static void main(String[] args) {
    Solution cr = new Solution();
    int[] ribbonLengths = {2,3,5};
    System.out.println(cr.countRibbonPieces(ribbonLengths, 5));
    ribbonLengths = new int[]{2,3};
    System.out.println(cr.countRibbonPieces(ribbonLengths, 7));
    ribbonLengths = new int[]{3,5,7};
    System.out.println(cr.countRibbonPieces(ribbonLengths, 13));
    ribbonLengths = new int[]{3,5};
    System.out.println(cr.countRibbonPieces(ribbonLengths, 7));
  }
}
```

This problem follows the `"Unbounded Knapsack"` pattern and is quite similar to `"Minimum Coin Change" (MCC)`. The only difference is that in MCC, we were asked to find the **minimum** number of coin changes, whereas, in this problem, we need to find the **maximum** number of pieces.

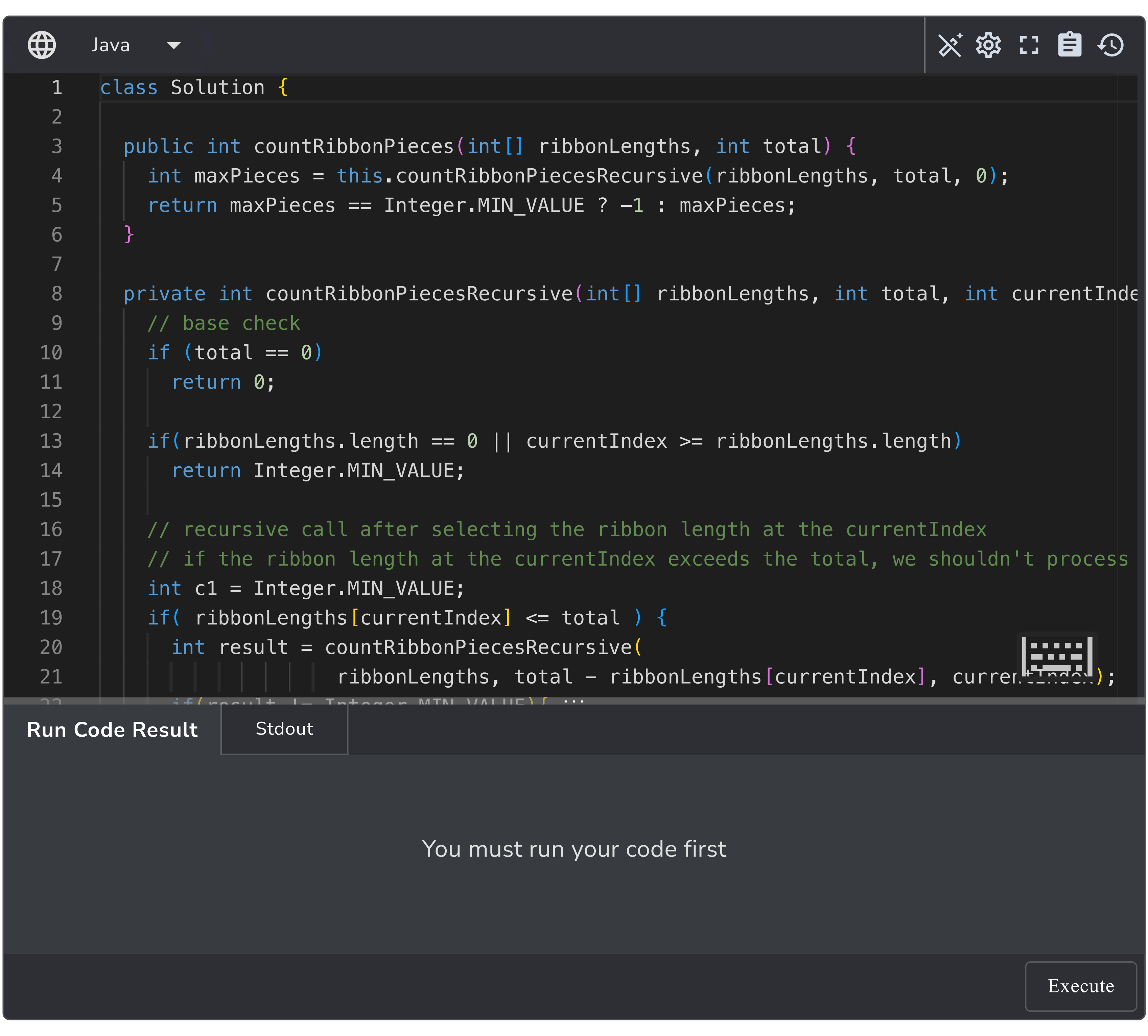
Basic Solution

A basic brute-force solution could be to try all combinations of the given lengths to select the maximum one that gives the total length of 'n.' This is what our algorithm will look like:

```
for each length 'l'
  create a new set which includes one quantity of length 'l' if it does not exceed 'n', and
     recursively call to process all lengths
  create a new set without length 'l', and recursively call to process the remaining lengths
return the number of pieces from the above two sets with a higher number of pieces
```

Code

Here is the code for the brute-force solution:



```java
class Solution {

  public int countRibbonPieces(int[] ribbonLengths, int total) {
    int maxPieces = this.countRibbonPiecesRecursive(ribbonLengths, total, 0);
    return maxPieces == Integer.MIN_VALUE ? -1 : maxPieces;
  }

  private int countRibbonPiecesRecursive(int[] ribbonLengths, int total, int currentInde
    // base check
    if (total == 0)
      return 0;

    if(ribbonLengths.length == 0 || currentIndex >= ribbonLengths.length)
      return Integer.MIN_VALUE;

    // recursive call after selecting the ribbon length at the currentIndex
    // if the ribbon length at the currentIndex exceeds the total, we shouldn't process
    int c1 = Integer.MIN_VALUE;
    if( ribbonLengths[currentIndex] <= total ) {
      int result = countRibbonPiecesRecursive(
              ribbonLengths, total - ribbonLengths[currentIndex], curren
```

The above algorithm's time complexity is exponential $O(2^{L+N})$, where 'L' represents total ribbon lengths, and 'N' is the total length that we want to cut. The space complexity will be $O(L+N)$.

Since this problem is quite similar to Minimum Coin Change, let's jump on to the bottom-up dynamic programming solution.

Bottom-up Dynamic Programming

Let's try to populate our array `dp[ribbonLength][total+1]` for every possible ribbon length with a maximum number of pieces.

So for every possible length 'len' (`0 <= len <= total`) and for every possible ribbon length index (`0 <= index < ribbonLengths.length`), we have two options:

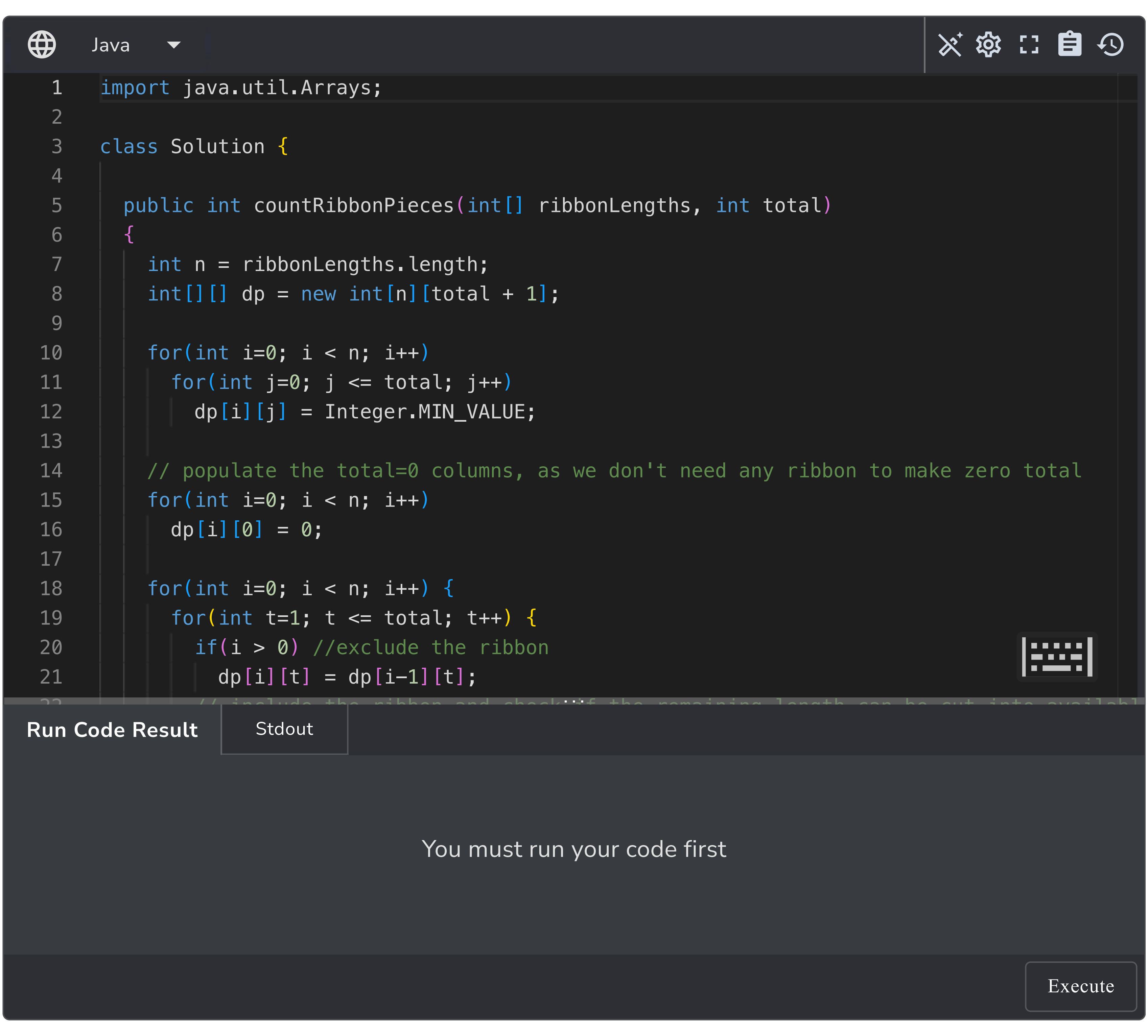
- **Exclude the ribbon length**: In this case, we will take the maximum piece count from the previous set => `dp[index-1][len]`
- **Include the ribbon length** if its value is not more than 'len': In this case, we will take the maximum pieces needed to get the remaining total, plus include '1' for the current ribbon length => `1 + dp[index][len-ribbonLengths[index]]`

Finally, we will take the maximum of the above two values for our solution:

```
dp[index][len] = max(dp[index-1][len], 1 + dp[index][len-ribbonLengths[index]])
```

Code

Here is the code for our bottom-up dynamic programming approach:



```java
import java.util.Arrays;

class Solution {

  public int countRibbonPieces(int[] ribbonLengths, int total)
  {
    int n = ribbonLengths.length;
    int[][] dp = new int[n][total + 1];

    for(int i=0; i < n; i++)
      for(int j=0; j <= total; j++)
        dp[i][j] = Integer.MIN_VALUE;

    // populate the total=0 columns, as we don't need any ribbon to make zero total
    for(int i=0; i < n; i++)
      dp[i][0] = 0;

    for(int i=0; i < n; i++) {
      for(int t=1; t <= total; t++) {
        if(i > 0) //exclude the ribbon
          dp[i][t] = dp[i-1][t];
```

The above solution has time and space complexity of $O(L * N)$, where 'L' represents total ribbon lengths and 'N' is the total length that we want to cut.

← Previous
Maximum Ribbon Cut

Next →
Fibonacci Numbers

☐ Mark as Completed